

DEEPSEEK HARNESS · ACCESSIBILITY LAYER

dsh-a11y

Follow a turn with your ears, not your mouse.

Coordinated announcements, keyboard navigation, and accessibility settings for the Harness Web UI.
Version 1.

POLITE
Tool bash started

ASSERTIVE
Approval requested

POLITE
Tool read finished

Coverage, not a rewrite.

`github.com/tespio/dsh-a11y`

THE SHORT VERSION

A narrator sitting next to the app

It watches state, decides what matters, and writes short messages onto two hidden noticeboards that screen readers watch.

1 Watch

App state: running tools, finished results, pending questions.

2 Decide

Is this worth saying? Is it a duplicate? Is it urgent?

3 Speak

Polite waits its turn. Assertive interrupts.

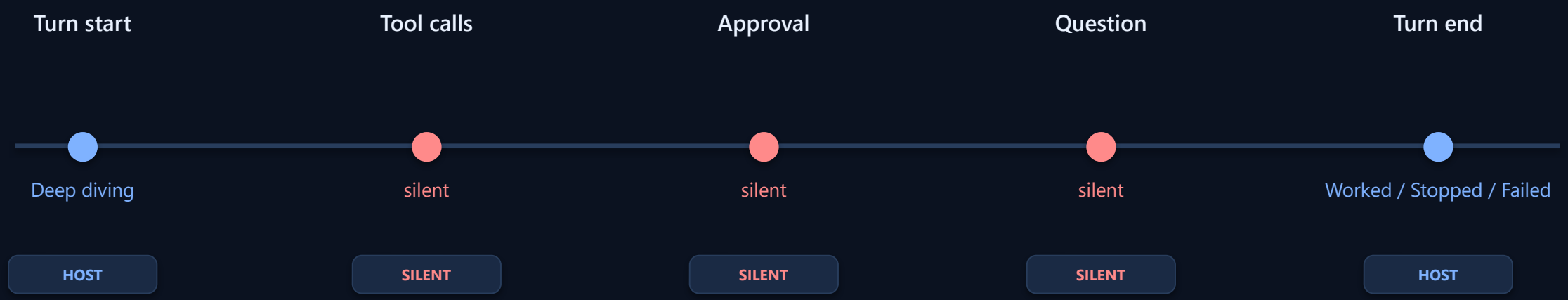
No DOM scraping. No second app. No rewriting the host.

The narrator is a coordinator. The noticeboards are live regions.

THE GAP

The host speaks at the edges. The middle is silent.

A screen-reader user hears the turn begin and end, but loses the thread in between.



Two live regions cover the edges. Nothing covers the work.

That is the gap dsh-a11y fills — without re-announcing the edges.

HOW IT WORKS

From state to speech

One coordinator. One queue. Two voices. The screen reader does the rest.

1
App state
useChat
useSessionStatus

2
Coordinator
decides and throttles

3
Queue
merges and paces

4
Live regions
polite + assertive

5
Screen reader
speaks the message

Polite
Waits for the screen reader to finish. Tool starts and finishes use this voice.

Assertive
Interrupts. Approvals and questions use this voice because the user needs to act.

Only one polite message is spoken at a time; the gap is a setting.

COVERAGE

What gets spoken

Everything in the middle of a turn. Nothing the host already owns.

Event	Spoken	Voice
Tool starts	“Tool bash started”	**Polite**
Tool finishes	“Tool bash finished”	**Polite**
Tool fails	“Tool bash failed”	**Polite**
Approval needed	“Approval requested for bash…”	**Assertive**
Question asked	“Question: Which browser?”	**Assertive**
Turn start / end	— host already speaks it	—

Detailed adds a short result and duration. Off silences the narrator; navigation still works.

THE #1 RISK

Don’t say it twice

Turn start and turn end stay with the host. The plugin owns the middle.

HOST OWNS THE BOUNDARIES

Turn start

“Deep diving”

Turn end

“Worked” / “Stopped” / “Failed”

`role=status` `aria-live=polite` `aria-atomic=true`

≠

DSH-A11Y OWNS THE MIDDLE

Tool lifecycle

`started / finished / failed`

Approvals

`requested from the tool`

Questions

`asked by the agent`

Verified in the shipped chat bundle: RunningStatus and TurnProcessNodeView are the only two turn-boundary live regions.

PACING

Fast tools shouldn’t create chatter

The queue merges a still-queued start into its finish, so one action is spoken once.

WITHOUT MERGING

Tool read started 0 ms

Tool read finished 60 ms

Two messages for one action. The second can even arrive before the first is spoken.

WITH MERGING

Tool read finished 60 ms

One message. The start was replaced before it reached the screen reader.

✓ **One action → one message**

250 / 700 / 1400 ms
announcement speed

30
max queued messages

ZWSP
so repeats are spoken

TRUST MODEL

Read state. Never scrape the screen.

The coordinator listens to the app’s own data, not to another plugin’s markup.

WHAT IT READS

`useChat`

running tool calls and finished tool results

`useSessionStatus`

a pending approval or question

State changes → decide → speak

WHAT IT NEVER DOES

- read another plugin’s DOM
- match hashed CSS classes
- copy page text into a live region
- watch the screen for changes

The only host DOM reads are focus targets: [data-approval-key] and [data-composer-input]. That’s focus management, not content.

NAVIGATION

Keyboard-first, without a rewrite

Four additions around the host’s existing controls.

Skip link

First Tab reveals “Skip to conversation”. Enter lands at the latest turn.

Turn headings

Every completed turn gets a hidden h2: “Turn 3: completed”. Jump between turns in the heading list.

Jump to latest

A compact button below the composer focuses the newest turn boundary.

Focus return

Approvals move focus into the card; answering restores it, or the composer.

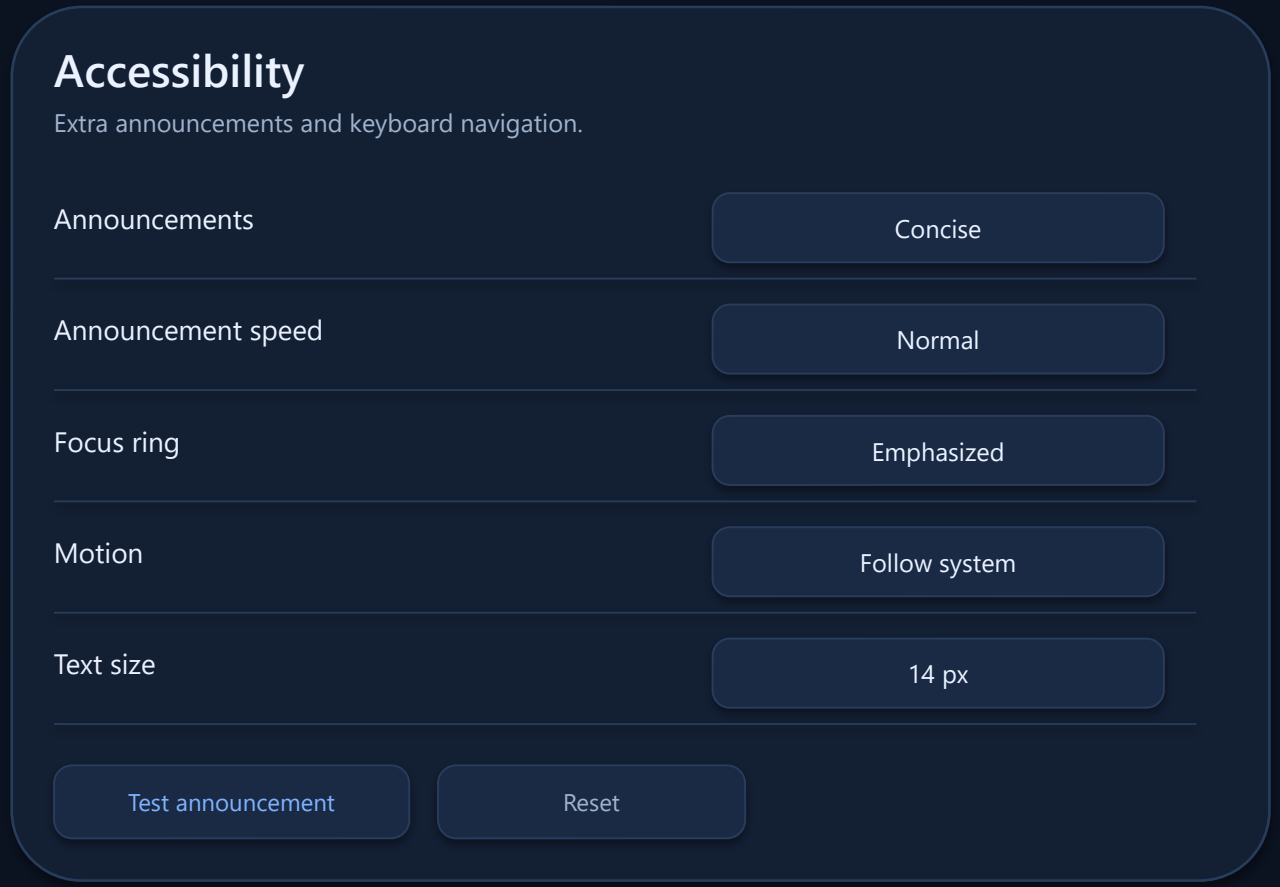
Focus-ring emphasis is a setting. The host settings modal already traps and restores focus; the plugin doesn’t add a second trap.

Known limit: there is no host slot at the start of the transcript, so skip targets and headings sit at turn tails.

SETTINGS

One page, host-native controls

A settings.section — not a new settings system.



- Stored in this browser (localStorage).
- Text size delegates to the host theme — the same as Appearance.
- Motion: follow system, force reduce, or force full.
- Focus ring: 3 px brand outline on every control.
- All colors use --dsw-alias-* tokens.

The page is additive: it sits beside Account, General, Models, Plugins, and Agent presets.

HONEST SCOPE

What it does not do

The plugin adds coverage. It is not a host rewrite and not a proof of speech.

No token-by-token speech

Streaming assistant text stays visual. Announcements are state transitions, not deltas.

No host markup fixes

Missing landmarks and inconsistent role=status usage are recorded as host findings, not patched with global CSS.

No v2 author kit yet

ctx.a11y, useFocusTrap, useRovingTabIndex, and auditSubtree are the next version.

No automatic screen-reader proof

Tests prove formatting, merging, and structure — not what Narrator actually says.

The skip target and turn headings live at turn tails because the host has no start-of-transcript slot.

VERIFICATION

What is proven, and what needs ears

The build is installed and live. Speech remains a human test.

PROVEN AUTOMATICALLY

- node test/run.mjs — formatting, merging, settings, locale parity
- React render smoke — skip link, live regions, headings, settings
- Apply smoke — six slot registrations and style injection
- Live inspect — all six registrations active after reload

NEEDS A HUMAN

- Narrator / NVDA speech and ordering
- axe-core over the app shell
- 200% zoom and 320 px viewport
- Real focus timing around approvals

No browser-control or screen-reader automation was available here. The manual protocol is in README.md.

ECOSYSTEM

The gap moved, but it didn’t close

The bare dsh-a11y name is still free on npm. Two other projects now cover adjacent ground.

`@oh-my-dsh/dsh-accessibility`

Diagnostics + Accessible View

A Settings → Accessibility diagnostic and a read-oriented Accessible View. Published versions target older DSH lines; its README says it cannot own live-region policy from outside the components.

`dsh-a11y-scan`

axe-core for local HTML

Scans local HTML, build output, or URLs and reports WCAG violations. A content scanner, not a UI live-announcement layer.

`dsh-a11y (this)`

Announcement coordinator

Targets the installed DSH 0.2.2-rc.1. Tool / approval / question announcements, skip and latest navigation, and the accessibility settings page.

dsh-a11y fills the middle: coordinated live announcements and keyboard navigation for the current DSH.

ROADMAP

v1 shipped. v2 is the author kit.

The coordinator is useful on its own. The kit makes every other UI plugin accessible by default.

V1 — SHIPPED

- Announcement coordinator
- Verbosity and speed
- Skip link, turn headings, jump to latest
- Focus ring and focus return
- Accessibility settings page
- Automated tests

V2 — NEXT

- ctx.a11y client service
- <LiveRegion> component
- useFocusTrap + useRovingTabIndex
- auditSubtree(ref) — own subtree only
- Audit panel for plugin authors
- Fixture-DOM audit tests

ctx.provide() is available to a Client half, so ctx.a11y is feasible. The hard rule stays: audit only the caller's own subtree.



DSH-A11Y

Coverage, not a rewrite.

Accessibility layer for the DeepSeek Harness Web UI

Coordinated announcements · keyboard navigation · accessibility settings

```
github.com/tespio/dsh-a11y
```

```
npm publish: next
```

Thank you.